*** Serialization & De-Serialization :

Serialization :

- It is a process to store the **Java objects into a file in binary Stream format.**

- In order to store the Objects into file in binary Stream, We can use a predefined class called ObjectOutputStream available in java.io package.

- ObjectOutputStream class has provided a predefined non static method called **writeObject(Object obj)**.

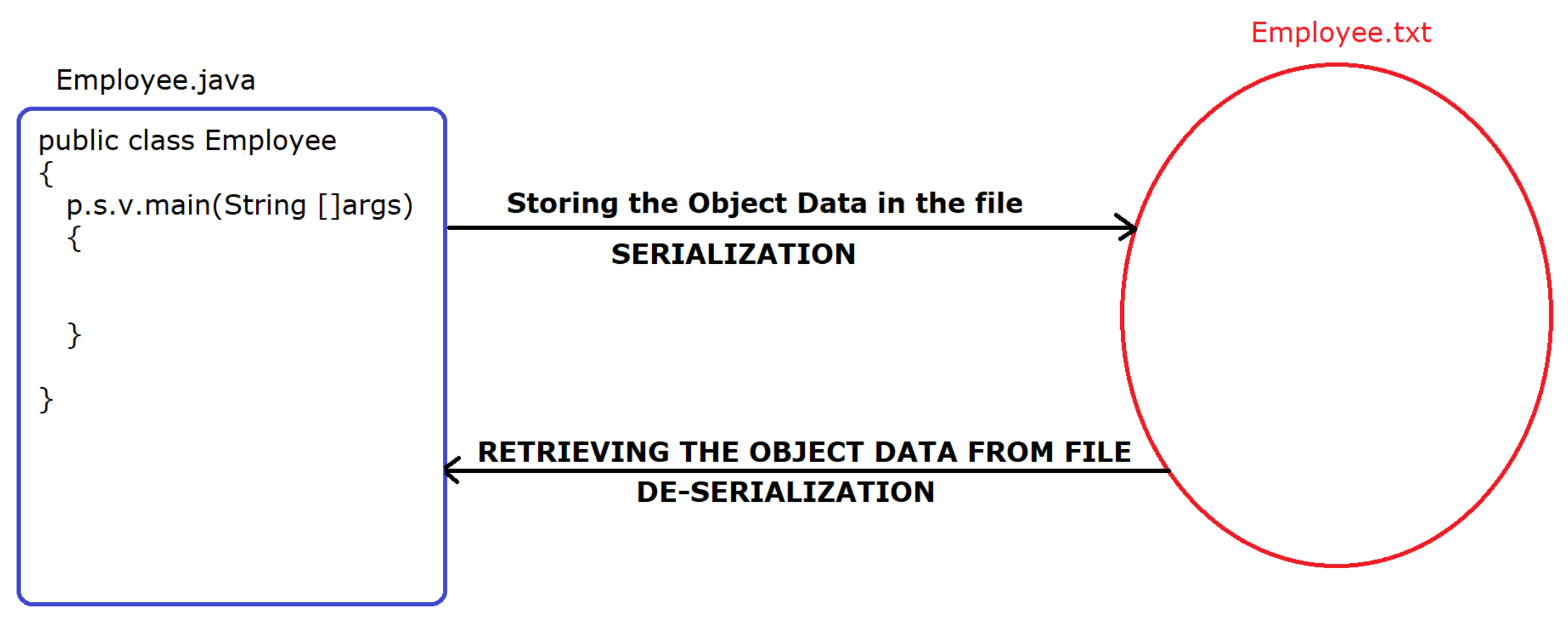
De-Serialization :

- It is the reverse process of Serialization.

- It is a process to **retrieve the Object data** from the file to Java application.

- In order to perform De-Serialization, Java has provided a predefined class called **ObjectInputStream available in java.io package.**

- **ObjectInputStream** class has provided a predefined non static method called **readObject(), return type of this method is Object.**


Employee.txt
Employee.java
public class Employee
{
p.s.v.main(String []args)
{
}
}
Storing the Object Data in the file
SERIALIZATION
RETRIEVING THE OBJECT DATA FROM FILE
DE-SERIALIZATION

Methods :

public final void writeObject(Object obj) throws IOException :

- It is a predefined non static method of ObjectOutputStream class, It is used to write the Object data to the file in binary Stream.

public final **Object** readObject() :

- It is a predefined non static method of ObjectOutputStream class, It is used to read the Object data from the file

- Performing serialization on any object is only allowed, If the corresponding object is Serializable Object that means the corresponsing class must implements from java.io.Serializable interaface.
  Example :
  If I want to store Employee object in a particular file then Employee class must implements Serializable interface

```
public class Employee implements java.io.Serializable
{

}
```

- If we are reading more than one object from a particular file then after reading all the objects, It will throw an exception i.e java.io.EOFException

---

//Program :

```java
package com.ravi.serialization_de_serialization;

import java.io.Serializable;

public class Employee implements Serializable
{
    //Version compatibility
    private static final long serialVersionUID = 1L;

    private int id;
    private String name;
    private double salary;
    private String address;

    private Employee(int id, String name, double salary, String address) {
        super();
        this.id = id;
        this.name = name;
        this.salary = salary;
        this.address = address;
    }

    //Static Factory Method
    public static Employee getEmployeeObject(int id, String name, double salary, String address)
    {
        return new Employee(id, name, salary, address);
    }


    @Override
    public String toString()
    {
        return "Employee [id=" + id + ", name=" + name + ", salary=" + salary + ", address=" + address +
"]";
    }
}
```

```java
 package com.ravi.serialization_de_serialization;

 import java.io.FileOutputStream;
 import java.io.IOException;
 import java.io.ObjectOutputStream;

 public class SerializationDemo {

     public static void main(String[] args) throws IOException
     {
         String filePath = "D:\\files\\Employee.txt";

         var fout = new FileOutputStream(filePath);
         var oos = new ObjectOutputStream(fout);

         try(fout; oos)
         {
             int noOfObjects = Integer.parseInt(IO.readln("Enter the no of objects :"));

             for(int i=1; i<=noOfObjects; i++)
             {
                 int id = Integer.parseInt(IO.readln("Enter Employee Id :"));
                 String name = IO.readln("Enter Employee Name :");
                 double salary = Double.parseDouble(IO.readln("Enter Employee Salary :"));
                 String address = IO.readln("Enter Employee Address :");
                 Employee emp = Employee.getEmployeeObject(id, name, salary, address);
                 oos.writeObject(emp);
             }

         }
         catch(IOException e)
         {
             e.printStackTrace();
         }
         catch(Exception e)
         {
             e.printStackTrace();
         }

         IO.println("Employee Object Stored successfully");
     }

 }
```

```java
package com.ravi.serialization_de_serialization;

import java.io.EOFException;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class DeSerializationDemo
{
    public static void main(String[] args) throws IOException
    {
        String filePath = "D:\\files\\Employee.txt";

        var fin = new FileInputStream(filePath);
        var ois = new ObjectInputStream(fin);

        try(fin; ois)
        {
            while(true)
            {
                  Employee employee =(Employee) ois.readObject();
                  IO.println(employee);
            }

        }
        catch(EOFException e)
        {
            System.err.println("End of file is reached "+e);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }

    }

}
```

---

transient keyword :

During serialization, If we don't want to serialized particular fields and want to provide default values at the time of De-Serialization then we should use transient keyword.
Example :

```
public class Customer
{
     private int id;
    private String name;
    private transient String address;  //address -> null
    private transient double balance;  //balance -> 0.0
    private long mobileNumber;
}
```

//Program :

```java
package com.ravi.serialization_de_serialization;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

class Customer implements Serializable
{
    private static final long serialVersionUID = 1L;

    private int id;
    private String name;
    private transient String address;  //address -> null
    private transient double balance;  //balance -> 0.0
    private long mobileNumber;

    public Customer(int id, String name, String address, double balance, long
mobileNumber) {
        super();
        this.id = id;
        this.name = name;
        this.address = address;
        this.balance = balance;
        this.mobileNumber = mobileNumber;
    }

    @Override
    public String toString() {
        return "Customer [id=" + id + ", name=" + name + ", address=" + address + ",
balance=" + balance
                + ", mobileNumber=" + mobileNumber + "]";
    }

}

public class SerAndDer {

    public static void main(String[] args) throws IOException, ClassNotFoundException
    {
        // Serialization

        var fout = new FileOutputStream("D:\\files\\Customer.txt");
        var oos = new ObjectOutputStream(fout);

        try(fout ; oos)
        {
            Customer customer = new Customer(101, "Scott", "Ameerpet", 120000,
9812345678L);
            oos.writeObject(customer);
            IO.println("Customer Object stored...");

        }
        catch(IOException e)
        {
            e.printStackTrace();
        }

        //De-Serialization
        var fin = new FileInputStream("D:\\files\\Customer.txt");
        var ois = new ObjectInputStream(fin);

        try(fin ; ois)
        {
            Customer customer = (Customer) ois.readObject();
            IO.println(customer);

        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }

}
```
